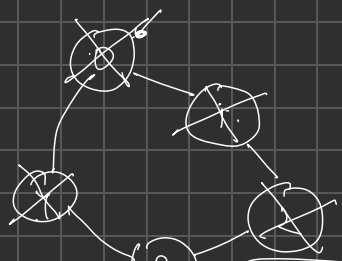
Rohit Negi

Circular game:

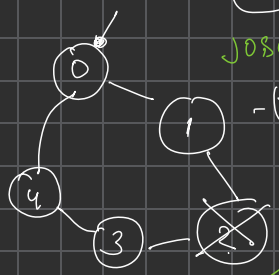
$n = 5$, $k = 3$



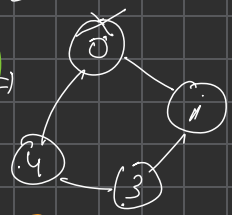
winner

josephus(n, k)

$n = 5$

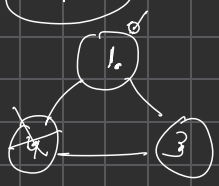


3 ⇒



⇒

$k = 3$



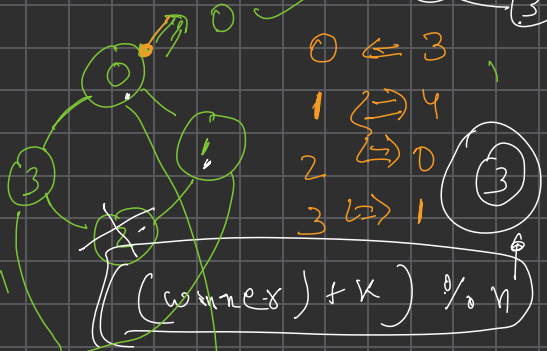
0

0 ⇐ 3

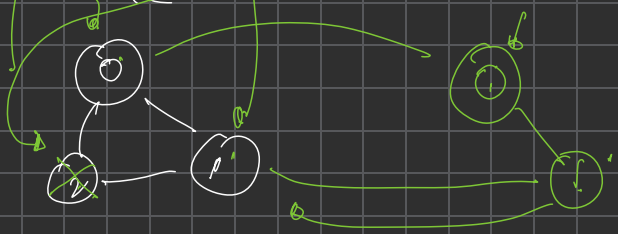
1 ⇐ 4

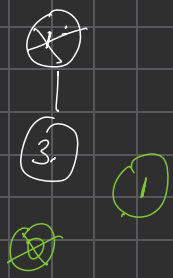
2 ⇐ 0

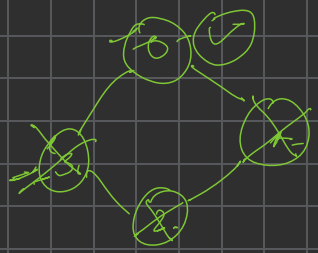
3 ⇐ 1

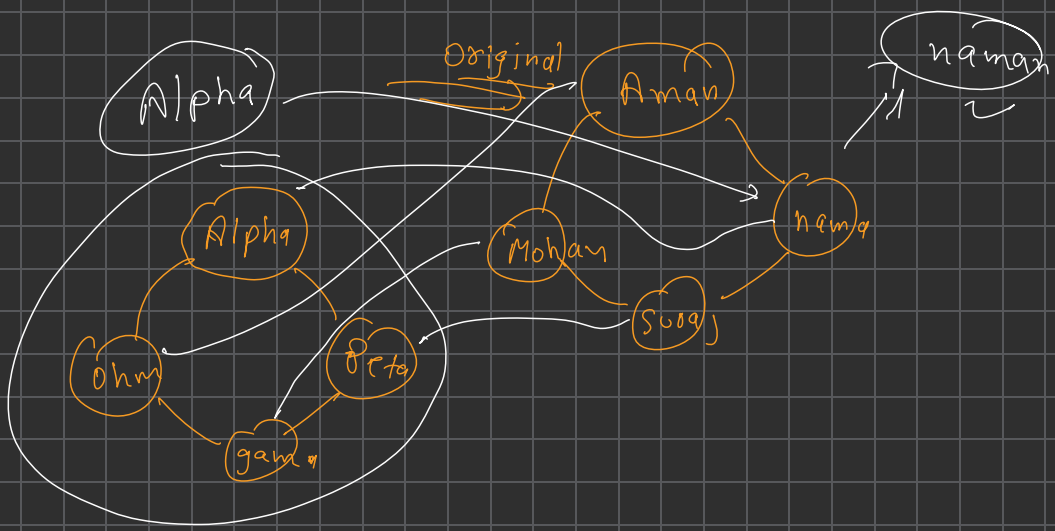


$(\text{winner} + k) \% n$








Alpha
Original
Aman
naman
Alpha
Mohan
nama
Ohm
Suraj
Beta
gama

(*) OOPS

(1) Student:

[name, age, rollno]

(2) 100 student

100 variable

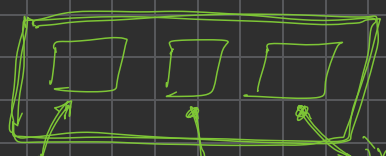
```
int main()
{
    String name = 'Rohit';
    int age = 15;
    int rollno = 120;

    String name2 = 'Mohan';
    int age2 = 10;
    int rollno2 = 70;

}
```

array

(1)

```
String name[100];
int age[100];
int rollno[100];
```

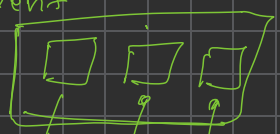
Student



⇒ Single Unit

int
float
bool

student

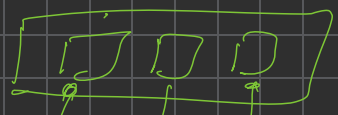


custom data structure

```
Class Student{
    String name;
    int age;
    int rollno;
}
```

// C++ code impl

int{ } ↓ defined



S1

stack

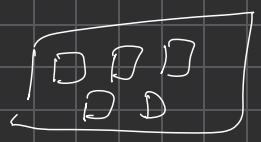
int a;

stack

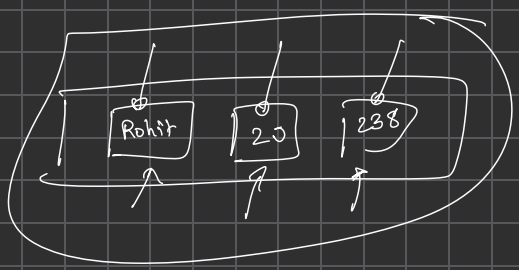
```
int main()
{
    Student S1;
    S1.name = "Rohit";
    S1.age = 20;
    S1.roll = 70;
}
```

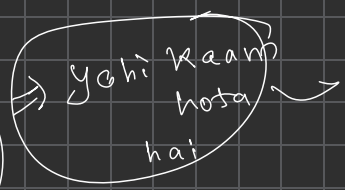
Bank Customer;

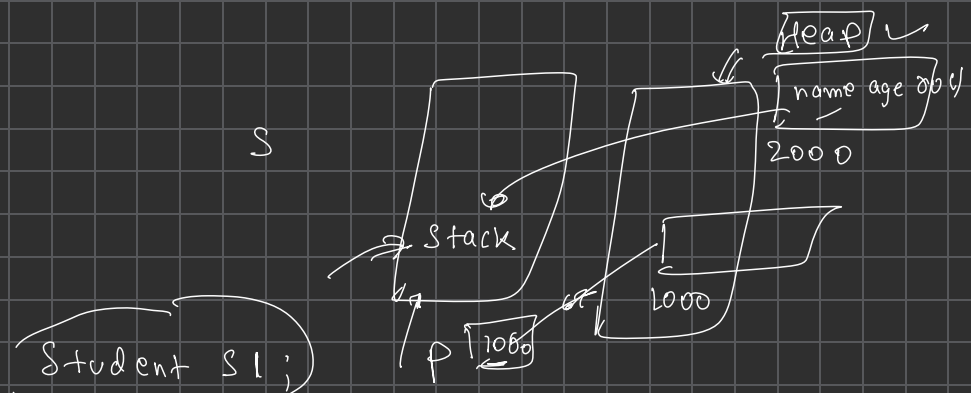
→ name, age, balance,
account_number, bank



```
Class Customer {

    String name;
    int age, balance,
    int accountNumber,
    String Bank,

}
```







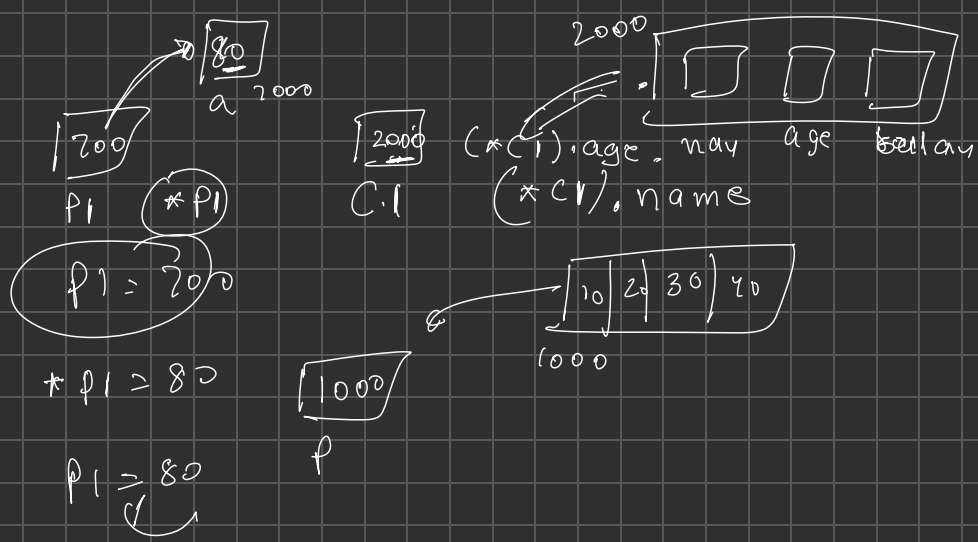
Student S1;

int arr [100]

[int * p ⇒ new int[10]]

Student * S1 ⇒ new Student("Rohit", 10, 3);

⇒ new Student();



p1 = 200

*p1 = 80

p1 = 80

[Linked List]

1 lakh

10	12	14	16	18
1000	1004	1008	1012	1016

address + 1 lakh

address + 5

value ⇒ Poor array

val	add
10	1400

1100

12	600

1400

14	900

600

16	1800

900

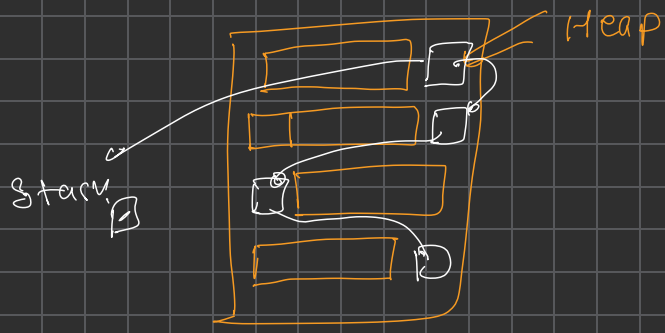
18	Null

1800

Stack

address = 1100

10 → 12 → 14 → 16 → 18



4 value
ko store
karna

1000 + 3 x 4

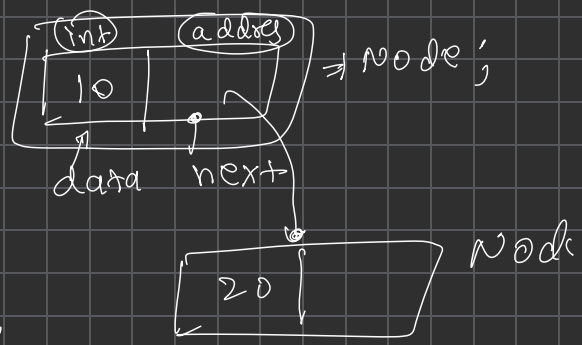
1012

0	1	2	3	5
10	20	30	40	50

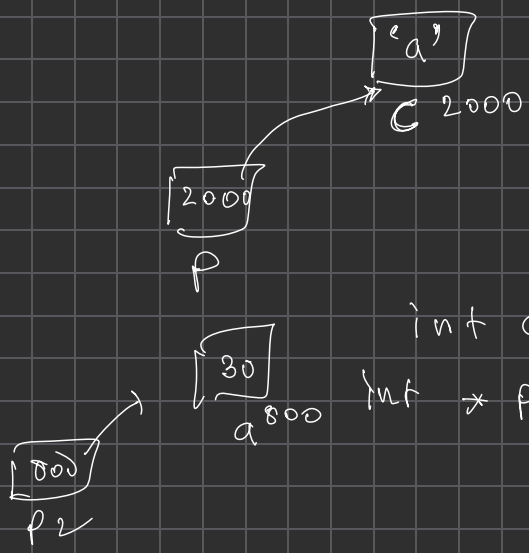
1000 9

10 → 20 → 30 → 40 → 50

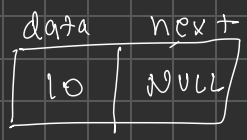
Linked List




```
Class Node
{
    Public;
    int data;
    Node * next;

}
```

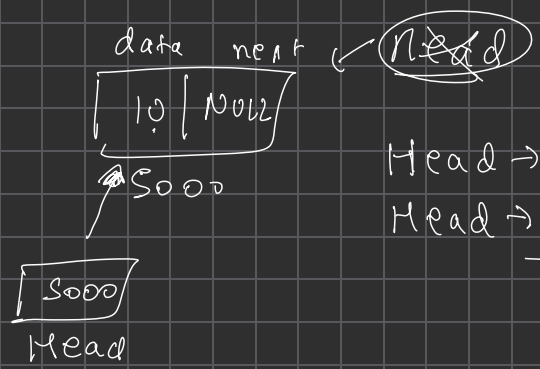



```
char c = 'a'
char * p = &c;

int a = 30;
int * p2 = &a;
```



Node N1(10, NULL);

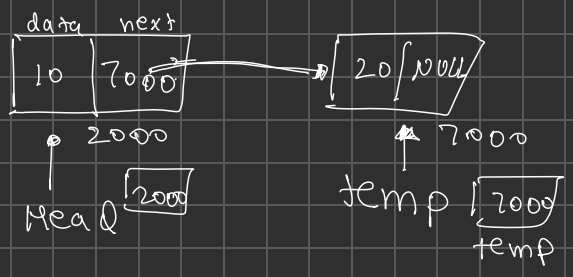
int a = 10 → stack



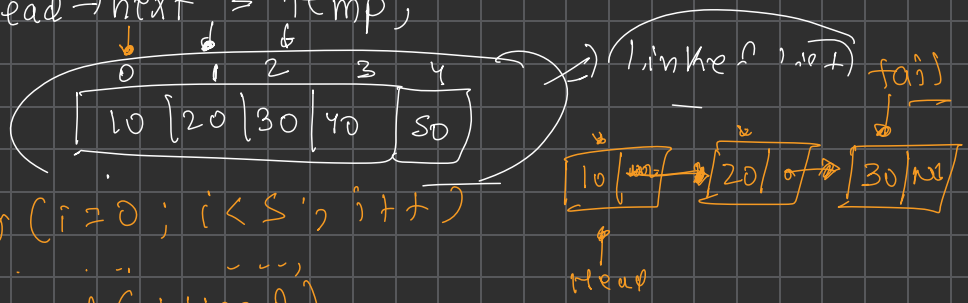
Head → data

Head → next

20

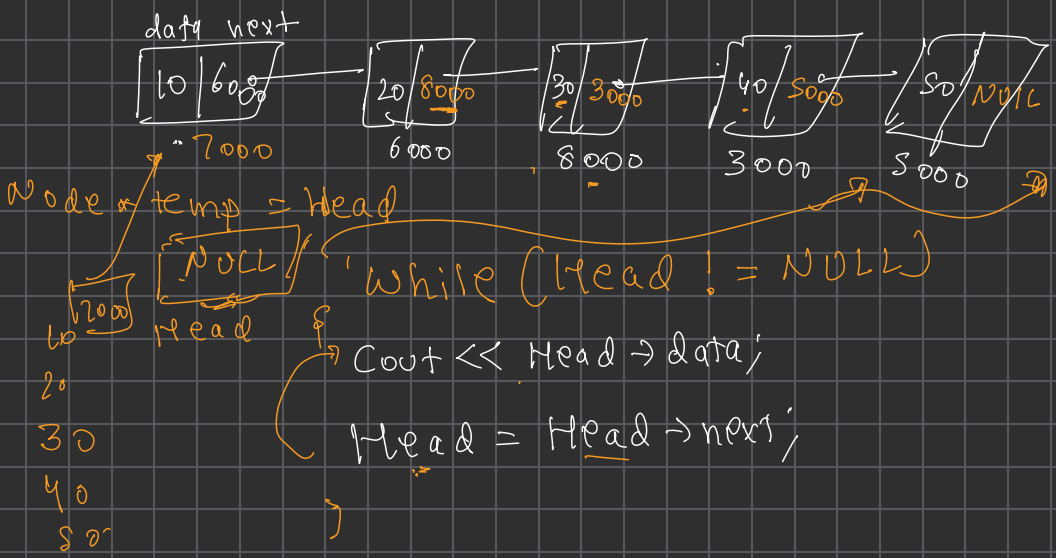



```
Node *temp = new Node(20, NULL)

Head->next = temp;
```




```
For (i=0; i<5; i++)
{
    if (!Head)
    {
        Head = new Node(arr[0], NULL);
        tail = Head;
    }
    else
    {
        tail->next = new Node(arr[i], NULL);
        tail = tail->next;
    }
}
```

Linked List

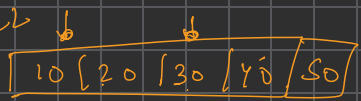


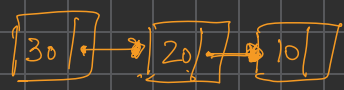

```
while (Head != NULL)
{
    Cout << Head -> data;
    Head = Head -> next;
}
```

Inserting element at end => Linked List:

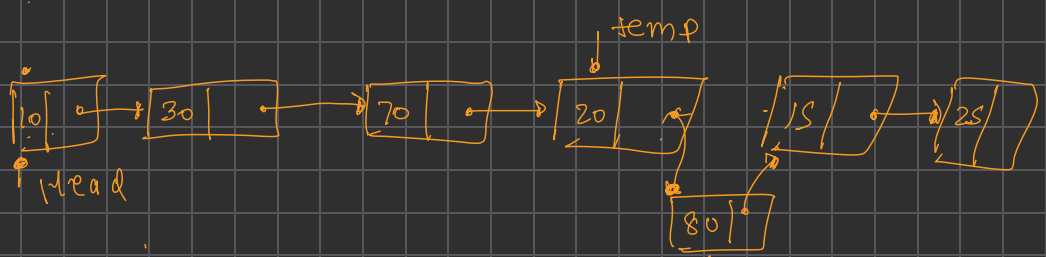
Inserting element at Beginning:

Node *Head = NULL





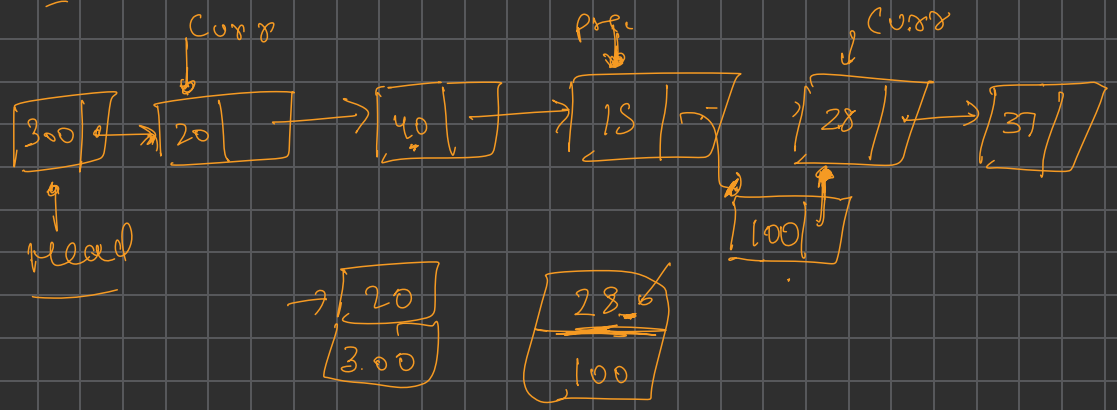

```
if(!Head)
{
    Head = new Node(10)
}
else
{
Node *temp = new Node(arr[i])
    temp->next = Head
    Head = temp;
}
```



data = 20 rakha

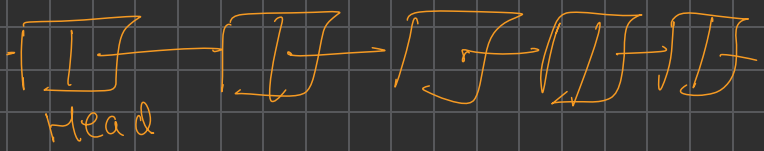
80

```
Node newNode = new Node(80);
newNode -> next = temp -> next;
temp -> next = newNode;
```




```
Node * newNode = new Node(100);
newNode -> next = curr
prev -> next = newNode;
```

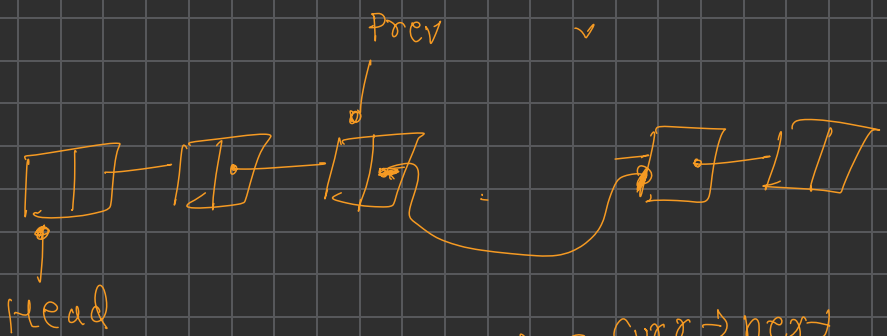
temp



(1) Delete ~~node~~ first nd

delete temp

```
while ( curr -> next != NULL )
{   Prev = Curr ;
    Curr = Curr -> next ;

}
delete curr ;
Prev -> next = NULL ;
```



Prev -> next = Curr -> next

Middle node — delete curr ;